

Token-Optimized MCP Gateway

Full architecture and build plan, in simple English

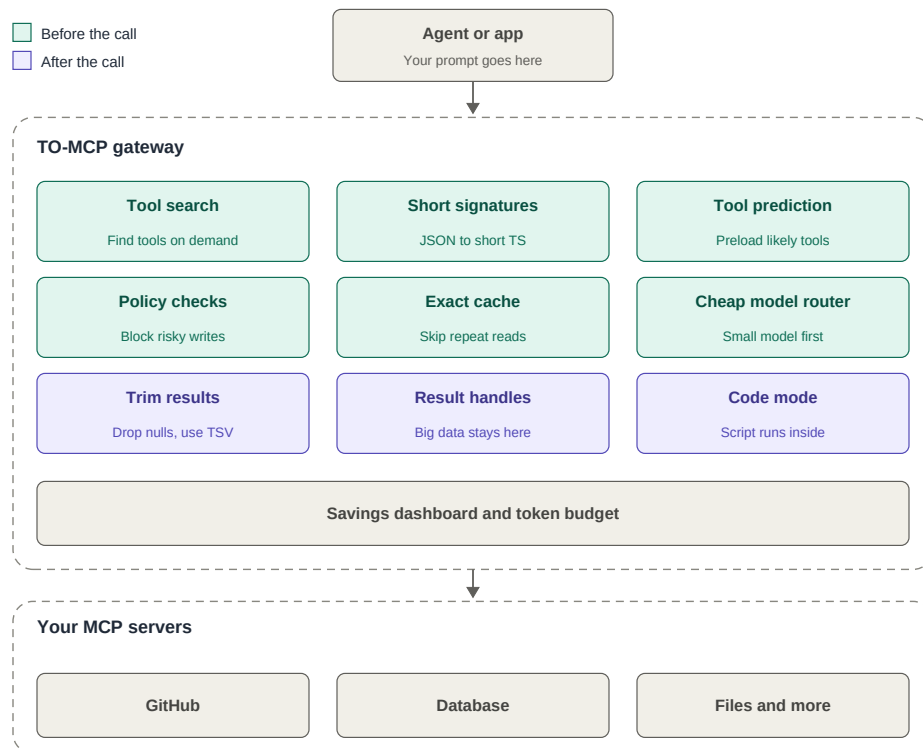
1. The problem

When an AI agent is connected to tools through MCP, every tool description is sent to the model before the work even starts. Every big tool result is also pushed back into the model's memory. Most of those tokens are never used, so everyone pays more and gets slower answers.

- **Tool bloat:** dozens of long JSON tool descriptions are loaded on turn 1, even if only two tools are used.
- **Result bloat:** a simple question can pull back a huge record with 80 fields.
- **Wasted spend:** the expensive model does easy work, and repeated calls are paid for again.

Goal: a small gateway that sits between the agent and the MCP servers and cuts tokens in both directions, so tool use becomes cheaper for everyone.

2. Architecture



The agent talks to one gateway, and the gateway talks to all your MCP servers. Teal techniques cut what goes in before a call. Purple techniques cut what comes back after a call.

3. The techniques, in simple English

Technique	What it does	What it saves
Tool search	The AI does not see every tool. It asks "which tool edits an issue?" and gets only the best matches.	Tool list tokens at start
Short signatures	Long JSON descriptions become one-line function signatures. If a tool is too complex to shorten safely, it keeps the full version.	Size of each tool description
Tool prediction	The gateway guesses likely tools from the first prompt and preloads them.	One search round trip
Policy checks	Read-only mode, confirmation before writes, allow and deny lists, and a warning on suspicious text in results.	Safety and trust
Exact cache	If the same read-only call was made recently, return the saved answer. Never used for write actions.	Repeat call tokens
Cheap model router	Easy steps like picking a tool or summarizing go to a small model. The strong model is kept for hard reasoning.	Cost per step
Trim results	Remove empty fields and show lists as compact tables (TSV).	Size of results
Result handles	A huge result stays inside the gateway. The AI gets a short preview and a handle, then asks only for the rows it needs.	Big result size
Code mode	The AI writes a small script that makes several tool calls inside the gateway. Only the final answer comes back.	Middle-step data
Savings dashboard	Shows tokens saved per tool and per session, with an optional budget limit.	Proves the value

4. Honest expectations

- Tool search and short signatures can cut tool description overhead by about 50 to 90 percent, mostly when many tools are mounted.
- Prompt caching already makes repeated tool descriptions cheaper, so claims like 97 percent total savings need testing against a cached baseline.
- Search adds a round trip, and weak search can pick the wrong tool. Always measure task success, not only tokens.
- Never serve cached answers for write actions, and avoid fuzzy cache matching for anything that changes data.

5. Build plan

Build one phase at a time with Claude Code. Do not move on until the checks pass.

Phase	What to build	Done when
1. Core gateway	Config loader, connect to downstream servers, catalog, tool search, short signature converter, and 3 meta-tools: <code>mcp_search_tools</code> , <code>mcp_call_tool</code> , <code>mcp_get_result</code> .	Works with 2 real servers. Client sees under about 400 tokens. Right tool in top 3 for 90 percent of test queries.
2. Result handling	Token estimate, null removal, TSV tables, store big results with a handle, slicing and grep, optional field projection.	A 50k-token result returns under about 1.5k tokens, and nothing is lost.
3. Stats	Track tokens used vs raw MCP baseline, CLI report, optional session budget.	Report numbers match recomputed logs.
4. Safety	Read-only mode, confirm writes, allow and deny lists, untrusted-text warning, secrets only from env.	Write actions cannot run without the right setting.
5. Benchmark	10 to 20 real tasks across 3+ servers, with and without the gateway, with caching on.	Honest report of tokens, cost, latency, and task success.
6. Advanced	Code mode sandbox, cheap model router, learned field pruning, tool prediction, exact-match cache for read-only tools.	Each feature shows a measured gain in the benchmark.

6. Suggested stack and rules

- TypeScript on Node 20 or newer, the official MCP SDK, MiniSearch for search, SQLite for stats and stored results, vitest for tests, zod for config.
- Everything runs locally. No cloud service and no hard-coded secrets.
- Work one phase at a time, write tests first for the signature converter and result store, keep commits small, and ask before adding dependencies.

7. First prompt for Claude Code

```
Read MCP_GATEWAY_PLAN.md. Initialize the TypeScript project, set up vitest, and implement Phase 1 step by step. Start with the config loader and the JSON Schema to short signature converter with tests, then the catalog, search, and the three meta-tools. Stop after the acceptance checks and summarize what passed.
```